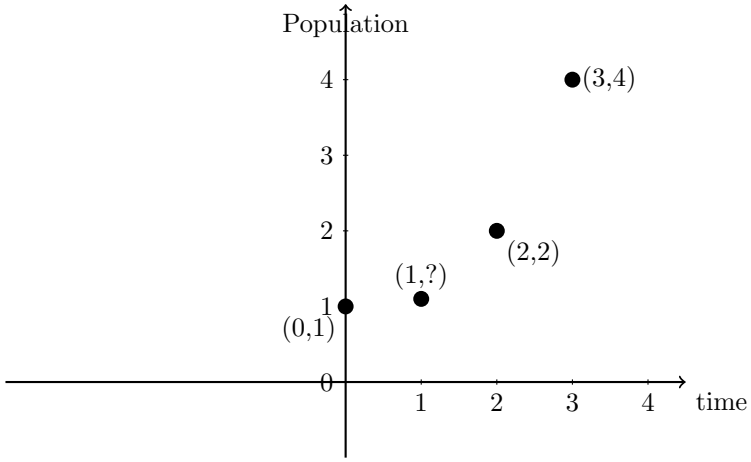
3 Interpolation

2024 Spring, UIC-DMS-NUM, by Xiaofeng Cai

Today, I start to talk about this topic, polynomial interpolation.

Motivation: Why do we study polynomial interpolation?

There are many answers for this question. For example, we have a set of data that is the population of a certain community as a function of time. Let's draw a graph to present this data.

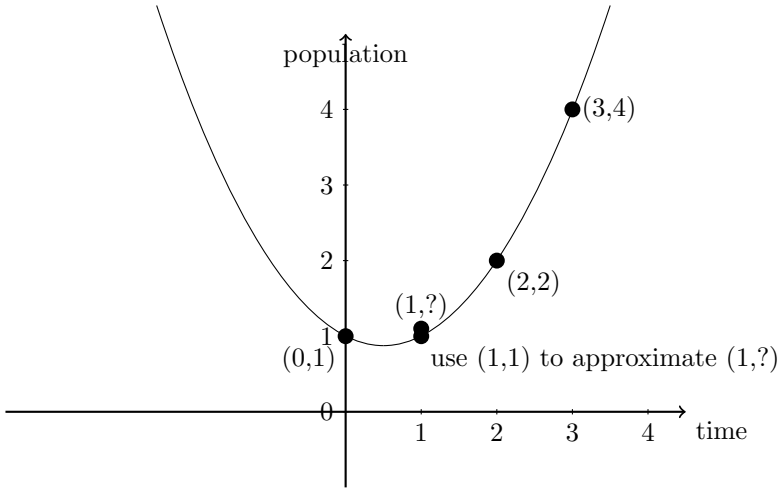


As you can see, we have recorded the data, $(0,1),(1,?),(2,2),(3,4)$ presented in the graph. We find that someone missed one data about year 1.

So a question arises:

Question 1. What can we do to know the population at year 1?

An idea for Question 1.: To construct a parabola by using these three points $(0,1),(2,2),(3,4)$.

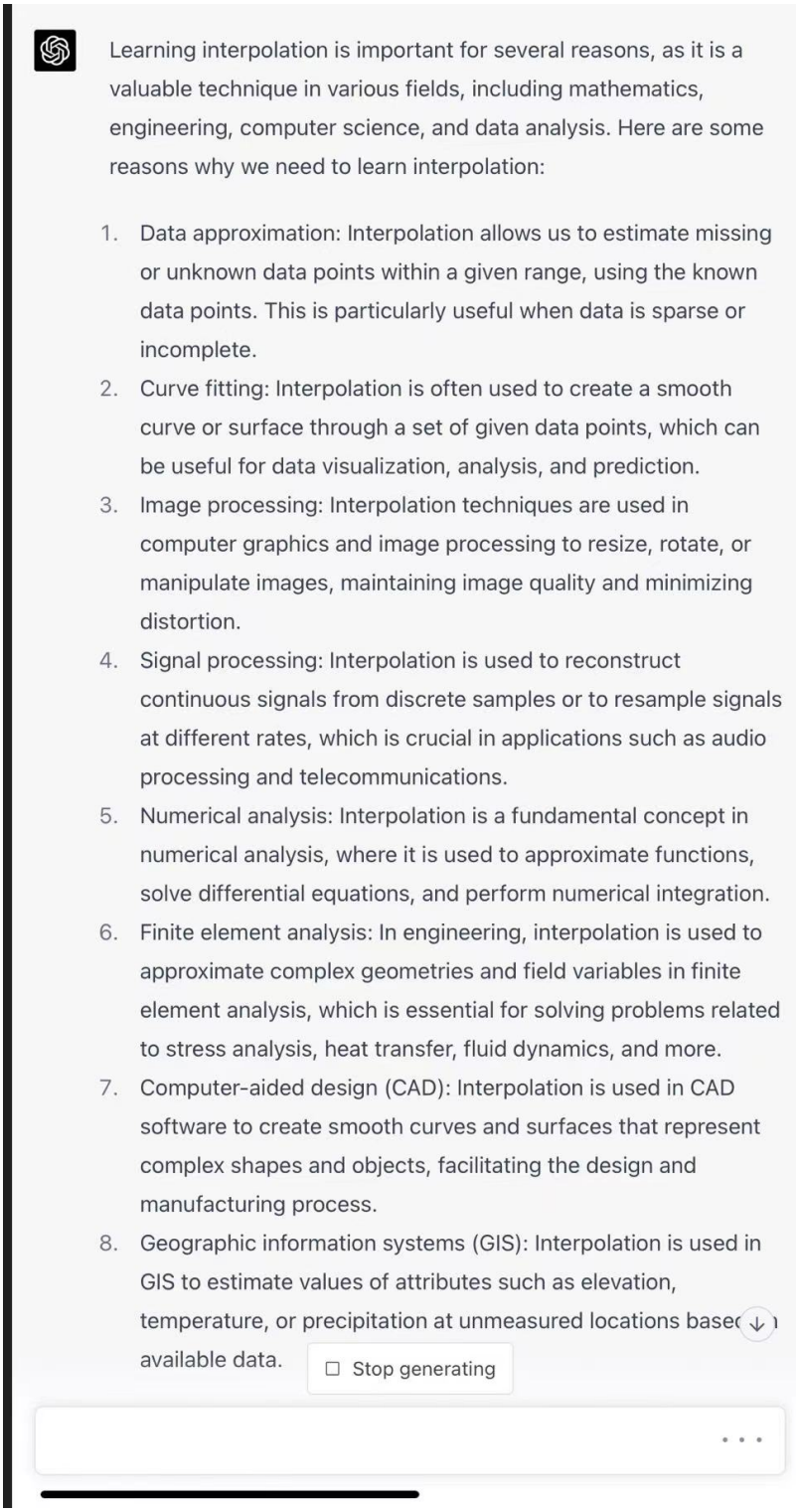


Now another equation arises:

Question 2. How to construct this parabola?

Answer to Question 2. In this lecture, I will introduce **polynomial interpolation**.

By ChatGPT, there are more examples in the following figure that show the importance of the interpolation.



Definition 3.1 The function $y = P_{n-1}(x)$ **interpolates** the data points $(x_1, y_1), \cdots, (x_n, y_n)$ if $P_{n-1}(x_i) = y_i$ for each $1 \leq i \leq n$.

Our goal: to find a polynomial of degree $n-1$, $P_{n-1}(x)$, such that interpolates the data points $(x_1, y_1), (x_2, y_2), \cdots, (x_n, y_n)$.

n **unknowns:** So $P_{n-1}(x)$ is a polynomial of degree $n-1$. Let's write it out the general form:

$$P_{n-1}(x) = c_0 + c_1 x + \cdots + c_{n-1}x^{n-1}.$$

So if you have a polynomial of degree $n-1$, there are n coefficients need to be known. So now you have a problem here in the end which contains n unknowns.

So what can we use to find these unknowns? We have n interpolation conditions.

n **conditions:** $P_{n-1}(x_i) = y_i$ for each $1 \le i \le n$.

So we have n equations that match the number of unknowns.

$$\begin{aligned} P_{n-1}(x_1) &= c_0 + c_1 x_1 + \cdots + c_{n-1}x_1^{n-1} = y_1 \\ P_{n-1}(x_2) &= c_0 + c_1 x_2 + \cdots + c_{n-1}x_2^{n-1} = y_2 \\ &\vdots \\ P_{n-1}(x_n) &= c_0 + c_1 x_n + \cdots + c_{n-1}x_n^{n-1} = y_n \end{aligned}$$

Can we write these three equations into a form of matrix vector $X\vec{c} = \vec{y}$?

Yes, we can. Here

$$\begin{bmatrix} 1 & x_1 & \cdots & x_1^{n-1} \\ 1 & x_2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^{n-1} \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-1} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

So if the data is given, we can solve it by the methods like Gaussian elimination we studied.

For example, given three points $(x_1, y_1), (x_2, y_2), (x_3, y_3)$, we can construct a parabola $P_2(x)$,

$$P_2(x) = c_0 + c_1 x + c_2 x^2$$

3 unknowns: c_0, c_1, c_2.

3 conditions: if $P_2(x_i) = y_i$, $i = 1, 2, 3$.

Then we have

$$\begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

So if the data is given, we can solve it by Gaussian elimination we studied.

Back to Example of the motivation. To find an interpolating for the data points $(0, 1), (2, 2)$ and $(3, 4)$ by solving a linear system.

Solution by solving a linear system: We can adopt the Gaussian elimination to solve the following system

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}.$$

Then we obtain

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 1 \\ -\frac{1}{2} \\ \frac{1}{2} \end{bmatrix}.$$

Therefore, we find $P_2(x) = 1 - \frac{1}{2}x + \frac{1}{2}x^2$.

And we know the population at year 1 am is $P_2(1) = 1$.

Check the interpolation conditions:

$$\begin{aligned} P_2(0) &= 1 \\ P_2(2) &= 1 - \frac{1}{2} \times 2 + \frac{1}{2} \times 2^2 \\ &= 2 \\ P_2(3) &= 1 - \frac{1}{2} \times 3 + \frac{1}{2} \times 9 \\ &= 4. \end{aligned}$$

It means that $P_2(x) = 1 - \frac{1}{2}x + \frac{1}{2}x^2$ really interpolates $(0,1), (2,2)$ and $(3,4)$.

Main Theorem of Polynomial interpolation: Theorem 3.2 Let $(x_1, y_1), (x_2, y_2), \cdots, (x_n, y_n)$ be n points in the plane with distinct x_i. Then there exists **one and only one** P of degree $n-1$ or less that satisfies $P(x_i) = y_i$ for $i = 1, 2, \cdots, n$.

Since the matrix of polynomial is a Vandermonde matrix whose determinant is nonzero, the conclusion can be true.

There are another "smart" ways to reconstruct the polynomial. An interpolating polynomial can be written in various forms, the most common being the **Lagrange form**, the **Newton form**, and **Neville form**. We begin with the Lagrange form since it is easier to understand. 3.1.1 Lagrange interpolation.

3.1.1. Lagrange interpolation Note that Lagrangian interpolation is an explicit formula. So now I first write down the Lagrange interpolation and explain why it can interpolates data points. To illustrate the idea behind the Lagrange interpolation, we consider three points for constructing $P_2(x)$.

$P_2(x)$**:** Suppose that we are given three points $(x_1, y_1), (x_2, y_2), (x_3, y_3)$. Then the Lagrange interpolating polynomial is

$$P_2(x) = y_1 \cdot L_1(x) + y_2 \cdot L_2(x) + y_3 \cdot L_3(x), \tag{1}$$

where

$$L_i(x) = \prod_{j=1, j\neq i}^{3} \left(\frac{x - x_j}{x_i - x_j} \right).$$

Specifically,

$$\begin{aligned} L_1(x) &= \frac{x - x_2}{x_1 - x_2} \frac{x - x_3}{x_1 - x_3}, \\ L_2(x) &= \frac{x - x_1}{x_2 - x_1} \frac{x - x_3}{x_2 - x_3}, \\ L_3(x) &= \frac{x - x_1}{x_3 - x_1} \frac{x - x_2}{x_3 - x_2}. \end{aligned}$$

We can see that $L_i(x)$ has the property of

$$L_i(x_j) = \begin{cases} 1 & i = j, \\ 0 & i \neq j. \end{cases}$$

Check the interpolation conditions:

$$\begin{aligned} P_2(x_1) &= y_1 \cdot L_1(x_1) + y_2 \cdot L_2(x_1) + y_3 \cdot L_3(x_1) \\ &= y_1 \cdot 1 + y_2 \cdot 0 + y_3 \cdot 0 \\ &= y_1 \\ P_2(x_2) &= y_1 \cdot L_1(x_2) + y_2 \cdot L_2(x_2) + y_3 \cdot L_3(x_2) \\ &= y_1 \cdot 0 + y_2 \cdot 1 + y_3 \cdot 0 \\ &= y_2 \\ P_2(x_3) &= y_1 \cdot L_1(x_3) + y_2 \cdot L_2(x_3) + y_3 \cdot L_3(x_3) \\ &= y_1 \cdot 0 + y_2 \cdot 0 + y_3 \cdot 1 \\ &= y_3 \end{aligned}$$

Back to Example of the motivation. To find an interpolating for the data points $(0, 1), (2, 2)$ and $(3, 4)$ by solving a linear system.

Solution by Lagrange interpolation: Substituting into Lagrange's formula (2) yields

$$\begin{aligned} P_2(x) &= 1\frac{(x-2)(x-3)}{(0-2)(0-3)} + 2\frac{(x-0)(x-3)}{(2-0)(2-3)} + 4\frac{(x-0)(x-2)}{(3-0)(3-2)} \\ &= \frac{1}{6}(x^2 - 5x + 6) + 2\left(-\frac{1}{2}\right)(x^2 - 3x) + 4\left(\frac{1}{3}\right)(x^2 - 2x) \\ &= \frac{1}{2}x^2 - \frac{1}{2}x + 1. \end{aligned}$$

Generally $P_{n-1}(x)$: Suppose that we are given n points $(x_1, y_1), (x_2, y_2), \cdots, (x_n, y_n)$. Then the Lagrange interpolating polynomial is

$$P_{n-1}(x) = y_1 \cdot L_1(x) + y_2 \cdot L_2(x) + \cdots + y_n \cdot L_n(x), \tag{2}$$

where

$$\begin{aligned} L_i(x) &= \prod_{j=1, j\neq i}^{n} \left(\frac{x - x_j}{x_i - x_j} \right) \\ &= \frac{(x - x_1)(x - x_2) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_1)(x_i - x_2) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)}. \end{aligned}$$

Example 3.2 Find the polynomial of degree 3 or less that interpolates the points (0,2),(1,1),(2,0), and (3,-1).

Solution: The Lagrange form is as follows:

$$\begin{aligned} P(x) &= 2\frac{(x-1)(x-2)(x-3)}{(0-1)(0-2)(0-3)} + 1\frac{(x-0)(x-2)(x-3)}{(1-0)(1-2)(1-3)} \\ &\quad + 0\frac{(x-0)(x-1)(x-3)}{(2-0)(2-1)(2-3)} - 1\frac{(x-0)(x-1)(x-2)}{(3-0)(3-1)(3-2)} \\ &= -\frac{1}{3}(x^3 - 6x^2 + 11x - 6) + \frac{1}{2}(x^3 - 5x^2 + 6x) - \frac{1}{6}(x^3 - 3x^2 + 2x) \\ &= -x + 2 \end{aligned}$$

Remark for Example 3.2 In Example 3.2, the data points are collinear, so the interpolating polynomial has degree 1. Theorem 3.2 implies that there are no interpolating polynomials of degree 2 or 3. It is clear to us that no parabola or cubic can pass through four collinear points.

3.1.2 Newton's divided difference

Now we start to study the Newton form of the interpolating polynomial. Again we will illustrate the idea behind the Newton form of the interpolating polynomial by constructing a parabola from three points.

$P_2(x)$: Suppose that we are given three points $(x_1, f(x_1)), (x_2, f(x_2)), (x_3, f(x_3))$. Using the data written in this order, the **Newton form of the quadratic interpolating interpolating polynomial** is

$$P_2(x) = b_0 + b_1(x - x_1) + b_2(x - x_1)(x - x_2).$$

In this case, the interpolating conditions are

$$\begin{aligned} P_2(x_1) &\equiv b_0 &&= f(x_1) \\ P_2(x_2) &\equiv b_0 + b_1(x_2 - x_1) &&= f(x_2) \\ P_2(x_3) &\equiv b_0 + b_1(x_3 - x_1) + b_2(x_3 - x_1)(x_3 - x_2) &&= f(x_3) \end{aligned}$$

or

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & (x_2 - x_1) & 0 \\ 1 & (x_3 - x_1) & (x_3 - x_1)(x_3 - x_2) \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \end{bmatrix}.$$

Note that the coefficient matrix of this system is lower triangular. When the nodes $\{x_i\}_{i=1}^3$ are distinct, the diagonal elements of the lower triangular matrix are nonzero. Consequently, the linear system has a unique solution which may be determined by forward substitution. In fact,

$$\begin{aligned} b_0 &= f(x_1) \\ b_1 &= \frac{f(x_2) - b_0}{x_2 - x_1} = \frac{f(x_2) - y_1}{x_2 - x_1} \\ b_2 &= \frac{f(x_3) - b_0 - b_1(x_3 - x_1)}{(x_3 - x_1)(x_3 - x_2)} \\ &= \frac{\frac{f(x_3)-f(x_1)}{x_3-x_2} - \frac{f(x_2)-f(x_1)}{x_2-x_1}\frac{x_3-x_1}{x_3-x_2}}{x_3 - x_1} \\ &= \frac{\frac{f(x_3)-f(x_2)+f(x_2)-f(x_1)}{x_3-x_2} - \frac{f(x_2)-f(x_1)}{x_2-x_1}\frac{x_3-x_2+x_2-x_1}{x_3-x_2}}{x_3 - x_1} \\ &= \frac{\frac{f(x_3)-f(x_2)}{x_3-x_2} + \cancel{\frac{f(x_2)-f(x_1)}{x_3-x_2}} - \frac{f(x_2)-f(x_1)}{x_2-x_1}\left(1 + \cancel{\frac{x_2-x_1}{x_3-x_2}}\right)}{x_3 - x_1} \\ &= \frac{\frac{f(x_3)-f(x_2)}{x_3-x_2} - \frac{f(x_2)-f(x_1)}{x_2-x_1}}{x_3 - x_1} \end{aligned}$$

Now we define the divided difference, which are the real numbers

$$\begin{aligned} f[x_k] &= f(x_k) \\ f[x_k, x_{k+1}] &= \frac{f[x_{k+1}] - f[x_k]}{x_{k+1} - x_k} \\ f[x_k, x_{k+1}, x_{k+2}] &= \frac{f[x_{k+1}, x_{k+2}] - f[x_k, x_{k+1}]}{x_{k+2} - x_k} \\ f[x_k, x_{k+1}, x_{k+2}, x_{k+3}] &= \frac{f[x_{k+1}, x_{k+2}, x_{k+3}] - f[x_k, x_{k+1}, x_{k+2}]}{x_{k+3} - x_k}, \end{aligned}$$

and so on.

With this notation, the **Newton form of the quadratic interpolating interpolating polynomial** is

$$\begin{aligned} P_2(x) &= b_0 + b_1(x - x_1) + b_2(x - x_1)(x - x_2) \\ &= f[x_1] + f[x_1, x_2](x - x_1) + f[x_1, x_2, x_3](x - x_1)(x - x_2) \end{aligned}$$

Newton form of $P_2(x)$: We summarize the procedure of the Newton form of the quadratic interpolating interpolating polynomial as follows:

1. *To write the Newton's divided difference triangle,*

$$\begin{array}{c|ccc} x_1 & f[x_1] & & \\ x_2 & f[x_2] & f[x_1, x_2] & \\ x_3 & f[x_3] & f[x_2, x_3] & f[x_1, x_2, x_3] \end{array}$$

2. *To read the coefficients of $P_2(x)$ from the top edge of the table,* we have

$$P_2(x) = f[x_1] + f[x_1, x_2](x - x_1) + f[x_1, x_2, x_3](x - x_1)(x - x_2).$$

Example 3.3 Use divided differences to find the interpolating polynomial passing through the points $(0, 1), (2, 2), (3, 4)$.

Solution: Applying the definition of divided differences lead to the following table:

$$\begin{array}{c|ccc} 0 & 1 & & \\ 2 & 2 & \frac{2-1}{2-0} = \frac{1}{2} & \\ 3 & 4 & \frac{4-2}{3-2} = 2 & \frac{2-\frac{1}{2}}{3-0} = \frac{1}{2} \end{array}$$

After completing the divided difference triangle, the coefficients of the polynomial $1, \frac{1}{2}, \frac{1}{2}$ can be read from the top edge of the table. The interpolating polynomial can be written as

$$P_2(x) = 1 + \frac{1}{2}(x - 0) + \frac{1}{2}(x - 0)(x - 2) = \frac{1}{2}x^2 - \frac{1}{2}x + 1,$$

matching the Lagrange interpolation version shown previously.

Newton form of $P_{n-1}(x)$: We summarize the procedure of the Newton form of the quadratic interpolating interpolating polynomial as follows:

1. *To write the Newton's divided difference triangle,*

$$\begin{array}{c|cccccc} x_1 & f[x_1] & & & & & \\ x_2 & f[x_2] & f[x_1, x_2] & & & & \\ x_3 & f[x_3] & f[x_2, x_3] & f[x_1, x_2, x_3] & & & \\ \vdots & \vdots & \vdots & f[x_2, x_3, x_4] & \ddots & & \\ x_{n-1} & f[x_{n-1}] & & \vdots & & & \\ x_n & f[x_n] & f[x_{n-1}, x_n] & f[x_{n-2}, x_{n-1}, x_n] & \cdots & f[x_1, x_2, \cdots, x_n] & \end{array}$$

2. *To read the coefficients of $P_2(x)$ from the top edge of the table,* we have

$$\begin{aligned} P_{n-1}(x) = f[x_1] &+ f[x_1, x_2](x - x_1) + f[x_1, x_2, x_3](x - x_1)(x - x_2) \\ &+ \cdots + f[x_1, x_2, \cdots, x_n](x - x_1)(x - x_2) \cdots (x - x_{n-1}). \end{aligned}$$

Example 3.4 Add the fourth data point $(1, 0)$ to the list in Example 3.3.

Solution: We keep the calculations that were already done and just add a new bottom to the triangle:

$$\begin{array}{c|cccc} 0 & 1 & & & \\ 2 & 2 & \frac{1}{2} & & \\ 3 & 4 & 2 & \frac{1}{2} & \\ 1 & 0 & \frac{0-4}{1-3}=2 & \frac{2-2}{1-2}=0 & \frac{0-\frac{1}{2}}{1-0}=-\frac{1}{2} \end{array}$$

The result is one new term to add to the original polynomial $P_2(x)$. Reading from the top edge of the triangle, we see that the new degree 3 interpolating polynomial is

$$P_3(x) = 1 + \frac{1}{2}(x-0) + \frac{1}{2}(x-0)(x-2) - \frac{1}{2}(x-0)(x-2)(x-3).$$

Note that $P_3(x) = P_2(x) - \frac{1}{2}(x-0)(x-2)(x-3)$, so the previous polynomial can be reused as part of the new one.

In summary, for the Newton form of the interpolating polynomial, it is easy to

- determine the coefficients.
- extend the polynomial to incorporated additional interpolation points and data.

Interpolation error

Theorem 0.1. *Assume that $P(x)$ is the (degree $n-1$ or less) interpolating polynomial fitting the n points $(x_1, y_1), \cdots, (x_n, y_n)$. The interpolating error is*

$$f(x) - P(x) = \frac{f^{(n)}(c)}{n!}(x - x_1)(x - x_2)\cdots(x - x_n), \tag{3}$$

where c lies between the smallest and largest of the number $x, x_1, \cdots, x_n$.

Runge phenomenon

Recall that we have studied interpolation polynomial. We know that we can use Lagrange interpolation or Newton interpolation. Please see the white-board about the process of Newton interpolation.

Specifically, we review Lagrange interpolation by solving a problem.

We can present what this polynomial looks like in Matlab.

m function

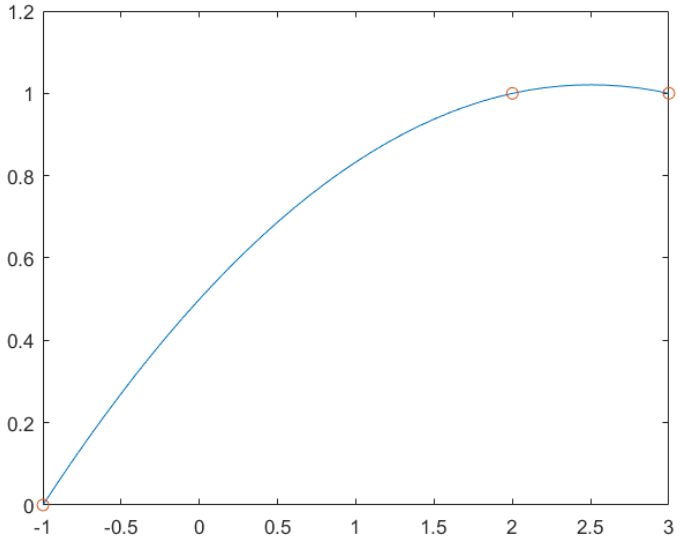
```
% Y=evaluatelagrange(x,y,X)
%
% Input:
% x : vector with n+1 entries (points x_i)
% y : vector with n+1 entries (points y_i)
% X : points where we want to evaluate
% Output:
% Y : values of the interpolant P at the points X
function Y=evaluatelagrange(x,y,X)
Y=0;
n=length(x)-1;
for k=1:n+1
  c=y(k);
  for j=[1:k-1 , k+1:n+1] % list of 1 to n+1 without k
     c=c.*(X-x(j))./(x(k)-x(j));
  end
  Y=Y+c;
end
return
```

plot it in Matlab

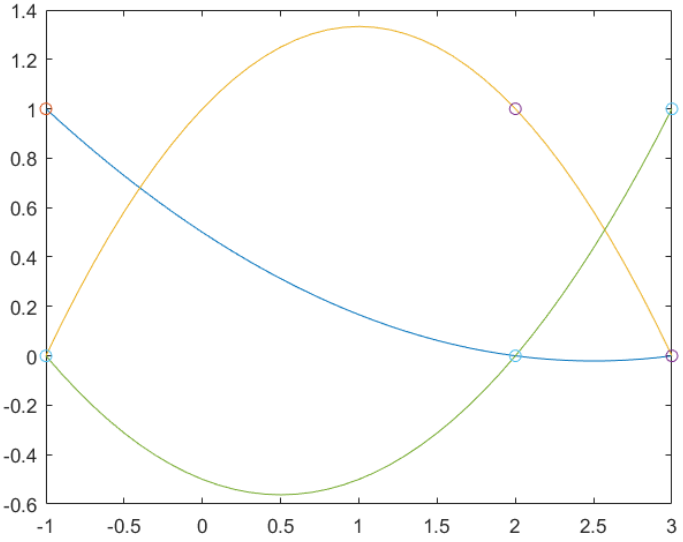
```
>> x=[-1 2 3];
>> y=[0 1 1];
>> xx=-1:0.1:3;
>> yy=evaluatelagrange(x,y,xx);
>> plot(xx,yy)
>> hold on
>> plot(x,y,'o')
```

What $L_1(x)$, $L_2(x)$ and $L_3(x)$ look like? `>> figure`

```
>> x=[-1 2 3];
>> y=[1 0 0];
>> xx=-1:0.1:3;
>> yy=evaluatelagrange(x,y,xx);
>> plot(xx,yy)
>> hold on
>> plot(x,y,'o')
>> hold on
>> x=[-1 2 3];
>> y=[0 1 0];
>> xx=-1:0.1:3;
>> yy=evaluatelagrange(x,y,xx);
```




```
>> plot(xx,yy)
>> hold on
>> plot(x,y,'o')
>> hold on
>> x=[-1 2 3];
>> y=[0 0 1];
>> xx=-1:0.1:3;
>> yy=evaluatelagrange(x,y,xx);
>> plot(xx,yy)
>> hold on
>> plot(x,y,'o')
```



It seems that if more points are used for interpolation, we can get better approximation. We claim that it might not always true and the so called "Runge phenomoenon" might be occurred for some problems.

An infamous example: Interpolate $f(x) = 1/(1 + 12x^2)$ at evenly spaced points in $[-1, 1]$.

- c = polyfit(x,y,n). Coefficients to the least-squares fitting polynomial with nodes (x,y). Vector c stores coefficients in descending order.
- y = polyval(c,xx). polynomial evaluation.

How to avoid Runge phenomenon

There are approaches:

- minimizing the interpolating error

$$f(x) - P(x) = \frac{f^{(n)}(c)}{n!}(x - x_1)(x - x_2)\cdots(x - x_n), \tag{4}$$

- piecewise continuous approximation
- piecewise discontinuous approximation

We consider the approach of the piecewise discontinuous polynomial approximation for two main motivations: one is that the functions in practice are always smooth enough; the other is that the functions are sometimes discontinuous.

Let us look at the first scheme.

Consider a function $f(x)$ in $[a, b]$.

The algorithm of the first scheme consists of the following steps:

Step 1 Partition: we cut $[a, b]$ into a set of n subintervals $\bigcup_{j=1}^{n}\left[x_{j-\frac{1}{2}}, x_{j+\frac{1}{2}}\right] := \bigcup_{j=1}^{n} I_j$; for simplicity, we consider a evenly spaced partition with $\Delta x = x_{j+\frac{1}{2}} - x_{j-\frac{1}{2}}$, $x_j = \left(x_{j-\frac{1}{2}} + x_{j+\frac{1}{2}}\right)/2$.

Step 2 In I_j, given $f(x_j)$. We directly use the constant $f(x_j)$ to present the function $f(x)$ in I_j.

We denote the approximating function as $\tilde{f}(x)$. This is the first time to illustrate what is the piecewise approximation. As you can see, it seems that we only do a partition and just use a function value to present the information in a subinterval; as a result, this algorithm is only first order of convergence rate, that is, $\|\tilde{f}(x) - f(x)\| = O(\Delta x)$.

In order to improve the accuracy of approximation, we introduce another scheme. Before we introduce the new scheme, we consider a function $f(x)$ in $[a, b]$ with the periodic boundary condition. The algorithm of this scheme consists of the following steps:

Step 1 Partition: we cut $[a, b]$ into a set of n subintervals $\bigcup_{j=1}^{n}\left[x_{j-\frac{1}{2}}, x_{j+\frac{1}{2}}\right] := \bigcup_{j=1}^{n} I_j$; for simplicity, we consider a evenly spaced partition with $\Delta x = x_{j+\frac{1}{2}} - x_{j-\frac{1}{2}}$, $x_j = \left(x_{j-\frac{1}{2}} + x_{j+\frac{1}{2}}\right)/2$. Besides, we can assume there are two ghost subintervals $I_0 = \left[x_{-\frac{1}{2}}, x_{\frac{1}{2}}\right]$ and $I_{n+1} = \left[x_{n+\frac{1}{2}}, x_{n+\frac{3}{2}}\right]$.

Step 2 In I_j, we use $(x_{j-1}, f(x_{j-1})), (x_j, f(x_j)), (x_{j+1}, f(x_{j+1}))$ to construct an interpolating polynomial $P_j^2(x)|_{I_j}$

Let us implement them for the following example.

Example 0.2. Consider $f(x) = \sin(\pi x)$ in $[-1, 1]$. We cut $[-1, 1]$ into n equal subintervals $\bigcup_{j=1}^{n}\left[x_{j-\frac{1}{2}}, x_{j+\frac{1}{2}}\right]$, $n = 4^{2^{k-1}}$, $k = 1, 2, 3$, where x_j is the center of $\left[x_{j-\frac{1}{2}}, x_{j+\frac{1}{2}}\right]$.

(a) Use the first scheme for approximating $f(x)$, and estimate the order of convergence.

(b) Assume that there is a periodic boundary condition; apply the above second scheme to approximate $f(x)$, and estimate the order of convergence.

Below are the reference code frames for this example.

```
clear

%%%%%%%%%%%problem setting
a = -1; b = 1;
```

```
f = @(x) sin(pi.*x)
%%%%%%%%%%%%end of problem setting

errori = zeros(1,3);
orderi = zeros(1,3);
for kk = 1:3
    figure
    n = 4*(2^kk);
    errori(kk) = 0.
    %partition
    xv = linspace(a,b,n+1)
    for i = 1:n
        xi(i) = ( xv(i)+xv(i+1) )/2.;
        fi(i) = f( xi(i) );
    end
    for i = 1:n
        %%%%%piecewise ploting
        xp = linspace( xv(i),xv(i+1) ,11 );
        yp = zeros(1,11);
        for k = 1:11
            yp(k) = polyinterp( [xi(i)],[fi(i)],xp(k) )
            errori(kk) = max( errori(kk), abs(yp(k)-f(xp(k))) )
        end
        plot(xp,yp,'r')
        hold on
        %

        %%%%%
    end
    if(kk>1)
        orderi(kk) = log2( errori(kk-1)/errori(kk) )
    end
end
```

Below is the code for problem (b).

```
clear

%%%%%%%%%%%%problem setting
a = -1; b = 1;
f = @(x) sin(pi.*x)
%%%%%%%%%%%%end of problem setting

errori = zeros(1,3);
orderi = zeros(1,3);
for kk = 1:3
    figure
    n = 4*(2^kk);
    errori(kk) = 0.
    %partition
    xv = linspace(a,b,n+1)
    dx = (b-a)/n
    for i = 1:n
        xi(i) = ( xv(i)+xv(i+1) )/2.;
        fi(i) = f( xi(i) );
```

```
    end
    for i = 1:n
        %%%%%piecewise ploting
        xp = linspace( xv(i),xv(i+1) ,11 );
        yp = zeros(1,11);
        for k = 1:11
            if(i==1)
                yp(k) = polyinterp( [xi(i)-dx xi(i) xi(i+1)],[fi(n) fi(i) fi(i+1)],xp(k) )
            elseif(i==n)
                yp(k) = polyinterp( [xi(i-1) xi(i) xi(i)+dx],[fi(i-1) fi(i) fi(1)],xp(k) )
            else
                yp(k) = polyinterp( [xi(i-1) xi(i) xi(i+1)],[fi(i-1) fi(i) fi(i+1)],xp(k) )
            end
            errori(kk) = max( errori(kk), abs(yp(k)-f(xp(k))) )

        end
        plot(xp,yp,'r')
        hold on
        %%%%%
    end
    if(kk>1)
        orderi(kk) = log2( errori(kk-1)/errori(kk) )
    end
end
```